

Halving the Troll Farm bot

Eight-hour engineering report: approaches, tests and current result

Work window	2 August 2026, 19:52 UTC to 3 August 2026, 04:37 UTC (approximately one continuous eight-hour engineering session)
Goal	Reduce the exact live bot from 62,820 bytes to at most 31,410 bytes, without hiding the logic through renamed variables, compression or unreadable minification, while keeping a mature Arena rank of 15 or better.
Current live bot	Unchanged: agent 6590141, submission 41081503, most recent recorded mature position rank 11 of 131.
Current verdict	The size goal has been reached repeatedly, but no half-size version has passed all behavior and safety checks. Nothing from this session was submitted to the Arena.

In one sentence. We can make the program half as large and eliminate its worst “walk back and forth forever” defect, but the reductions still create rare, very expensive game losses; the newest evidence points to **when the apple orchard is activated**, not to the orchard’s mere existence, as the next place to simplify carefully.

1. What game problem are we solving?

In Troll Farm, a program controls trolls on a grid. Trolls move, collect fruit, mine iron, chop trees, plant fruit and bring resources back to their tent. The bot must coordinate its workers: one blocked doorway or one bad planting decision can waste many turns. Fruit has value, but wood is especially important for the final score. A good policy therefore has to balance short-term collection, training a second troll, denying resources to the opponent, creating a renewable orchard and converting remaining fruit into wood before the game ends.

The live program, called **E7a** in the project, is already competitive. Its latest mature checkpoint was rank 11. The engineering challenge is deliberately harder than ordinary code cleanup: remove half of the source while preserving the decisions that made it competitive. The target is exactly half of 62,820 bytes, so a qualifying source may contain at most 31,410 bytes.

This was a logical simplification exercise. Variable names were not shortened to meaningless letters, the source was not compressed, and formatting was not crushed merely to save bytes. Each candidate was built by deleting unused generality or replacing a large decision block with a smaller, named policy that can still be read and reviewed.

2. How the tests work, in plain language

Every candidate was compared with the exact live bot on the same generated maps, against the same opponents, and from both starting sides. This paired comparison removes much of the noise: if both programs receive the same difficult map, their difference is more informative than either raw score alone.

Term used in the records		Meaning
Baseline		The exact 62,820-byte live E7a program used as the reference.
Candidate		A proposed smaller program being tested against the baseline. Iteration labels such as “r5” or “r32” are simply revision numbers, not new algorithms by themselves.
Paired margin change		For one game, first calculate “our score minus opponent score” for each program, then subtract the baseline result from the candidate result. Positive is better.
Development panel		A reusable collection of known maps used to diagnose and improve code. Passing it is encouraging, but it cannot prove that the program generalizes because engineers have seen its results.
Untouched or fresh panel		A set of map numbers that is locked before any map is generated or inspected. It is run once. Its result is final for that exact source, preventing repeated tuning until a lucky test appears.
Opponent family		One of six fixed opponent programs with different strategies. Requiring success across families prevents a candidate from improving only against one convenient style.
Bootstrap bound	lower	A conservative estimate obtained by resampling whole maps 50,000 times. It asks whether the average improvement might be an accident of the particular maps selected.
Catastrophe		A game lost by at least 100 points. The candidate may not create more catastrophes than the baseline.
Negative-margin mass		The total size of all losses. A 200-point loss counts more than a 5-point loss. This catches a candidate that wins often but occasionally collapses.
Period-2 oscillation		A troll alternates between two positions, A-B-A-B, instead of completing its job. A run of six or more alternating moves is treated as a serious liveness defect.
Transfer		Evidence that a policy which looked good on development maps also works on unseen maps.

The full panel contains 43 maps, two starting sides and six opponent programs: 516 paired games. Other checks verify compilation, legal command generation, second-troll training, execution time, empty-input handling and replay of known oscillation examples. A candidate must pass *all* frozen gates; a good average cannot excuse a crash or a worse catastrophic-loss tail.

3. The work in chronological order

3.1 First cut: prove half-size is mechanically possible

The initial byte audit identified large blocks that were general but unnecessary for this live two-worker policy: support for arbitrary worker counts, unused protocol and trait machinery, generic target containers, unused runtime state and broad fallback policies.

The first integrated cut, revision 5, was only 30,949 bytes and compiled successfully. It was a useful mechanical proof, but a behavioral disaster: over a 16-game smoke test it lost 262.5 points more than the baseline on average and produced 14 catastrophes. Restoring economic forecasting and coordination repaired revision 18 to only 6.6 points worse, but it had grown to 35,146 bytes—3,736 bytes over the limit. This established the central trade-off: most of the original size was tied to behavior that really mattered.

3.2 A size-qualified version that removed too much

Revision 32 was 31,387 bytes, 23 bytes under the ceiling. It compiled, passed ten fixed semantic examples and moved acceptably in small tests. The full 516-game development panel exposed the failure: average paired result -53.66 , catastrophes 19 to 64, and total loss mass 4,138 to 15,143. All six opponent styles became worse. Small smoke tests had been too narrow.

Block-by-block diagnosis showed two important facts. First, deleting the orchard alone already cost 7.64 points on average even though the remaining source was still 48,644 bytes. Second, keeping the exact economic core but placing it under a simplified outer policy cost 27.45 points at 33,167 bytes. Restoring the exact tuned opening recovered much of the loss, to -9.81 , but raised size to 36,059 bytes. The opening and orchard were therefore not disposable decoration.

3.3 Structural specialization instead of blunt deletion

The next approach preserved the valuable opening and economics while specializing the program to what the live bot actually does: two workers, a smaller set of targets, fewer zero-value branches and less unused state. A 31,337-byte version passed the development panel with average $+5.53$, conservative lower bound $+1.82$, catastrophes improving 19 to 11, and no long two-cell oscillations.

Its one-shot unseen test rejected it. The average remained positive at $+3.30$, but the result was map-fragile and total loss mass increased from 4,385 to 4,891. This was the first strong warning that the remaining problem lived in rare tail events rather than in average play.

3.4 Repairing tent traffic and orchard geometry

Replay traces found two concrete coordination failures:

- A faster rear troll carrying wood could pin the front carrier near the tent. The repair gave the front carrier priority so the pair moves like an ordered convoy.
- Starting an orchard could consume too much access space around the tent. The repair allowed a mixed-use tent door only when a third usable door remained free.

The resulting 31,398-byte candidate passed development strongly and then passed its untouched 516-game panel: average $+5.55$, lower bound $+1.29$, fewer catastrophes and lower loss mass. It looked ready—until exact replays of real Arena games exposed an empty-selection crash and, after the crash was made safe, long two-cell oscillations in 10 of 25 known bad games. A program that can freeze a troll is not safe to deploy even when aggregate scores are good, so this candidate was superseded.

3.5 Stopping repeated backtracking

The next family of candidates remembered recent positions. If a newly chosen move would recreate an A-B-A-B pattern, the movement resolver blocks or redirects it. This made every tested half-size candidate liveness-clean: the fresh panels went from more than 100 long oscillation episodes in the baseline to zero.

The first guarded version was 31,405 bytes. It also deleted three pieces of logic to pay for the new memory: speculative reservation of tent stock, part of a rule for leaving a fully funded tent, and an occupied-door prefilter. It was excellent on development maps ($+9.45$ average, $+4.04$ lower bound) and on unseen maps ($+9.46$, $+1.74$ lower bound). Nevertheless, the strict tail gate failed by one: catastrophes changed from 26 to 27. Later replay attribution showed that the dangerous deletion was the simplified rule for evacuating a fully funded tent, not the stock-reservation deletion.

3.6 Preserve tent evacuation; simplify the oscillation guard

A 31,248-byte successor restored the original tent-evacuation rule and replaced the larger oscillation detector with a compact rule: do not immediately move back to the previously observed cell. It passed development at +9.14 with fewer catastrophes and no long oscillations. On its fresh panel it remained positive at +3.92 and improved the loss tail, but it failed breadth: the average was negative against two of the six opponent styles. That source was rejected without Arena submission.

Traces then showed why an unconditional “never step back” rule is too coarse. Some harmful reversals occur at a tree, where changing direction abandons work. Other reversals happen on an open route and are legitimate corrections. Thresholds based on worker role, worker count or opponent workforce did not separate these cases reliably.

3.7 Tree-aware backtracking: the newest exact candidate

The newest source blocks the second consecutive reversal when either end of the move is a tree, but permits a bounded correction on an ordinary open route. This rule is called the “tree-edge reversal” policy in internal filenames: an *edge* simply means the move between two adjacent map cells.

The source is 31,407 bytes—three bytes below the target—and remains readable. It passed the full development panel at +8.22 average, +3.02 lower bound, catastrophes 19 to 12, and zero long oscillations. On the one-shot unseen panel it still averaged +6.29 and passed 11 of 13 gates, but catastrophes worsened 12 to 16 and total loss mass worsened 4,567 to 4,826. It was therefore rejected and not submitted.

4. Condensed result table

Stage	Bytes	Main result	What it taught us
First cut (r5)	30,949	✗ −262.5 in smoke test	Half-size compilation is easy; preserving behavior is not.
Repaired cut (r18)	35,146	Behavior mostly repaired; too large	Economic forecasting and coordination consume real bytes.
Revision 32	31,387	✗ −53.66 on 516 games	Small smoke tests cannot protect against broad strategic collapse.
Structural specialization	31,337	✓ development; ✗ fresh loss tail	Named specialization works, but unseen-map tail risk remains.
Convoy + safer orchard	31,398	✓ fresh scores; ✗ crash / liveness replay	Aggregate value gates are insufficient without exact replay safety.
Full oscillation guard	31,405	✗ one extra catastrophe	A tent-evacuation simplification caused a rare collapse.
No-immediate-backtrack	31,248	✗ two opponent styles worse	Global movement bans also block legitimate route corrections.
Tree-aware backtrack	31,407	✗ catastrophe and loss-mass gates	The movement rule is better, but a separate tail mechanism remains.

5. Latest diagnosis after the newest rejection

The rejected fresh maps can still be replayed for diagnosis, but they can never qualify another source. Five controlled variants were compared on the exact same 516 preserved rows.

Variant, explained	Bytes	Mean	Cats.	Loss mass
Strictly forbid immediate backtracking	31,248	+6.41	19	5,017
Allow one bounded correction over five moves	31,394	+6.22	16	4,823
Block repeated reversal at a tree edge	31,407	+6.29	16	4,826
Over-limit control retaining more exact tent logic	31,848	+6.37	19	4,928
Retain stock reservation, but with a liveness flaw	31,394	+6.53	17	4,776
Remove the compact orchard completely	28,517	−38.72	48	15,719

Here “Cats.” is the number of at-least-100-point losses; the baseline had 12, and baseline loss mass was 4,567. These diagnostic numbers show that the newest fresh failure is *not* caused solely by the tree-aware movement rule, stock reservation, or the smaller tent-door logic. The tree-aware rule is actually better than the strict rule on the tail. They also show that globally deleting the orchard is disastrous, despite saving almost 2.9 kilobytes beyond the required target.

The worst fresh task was examined command by command. It used map number 9,866,014, our first starting side, and the adaptive gold-strength opponent. The baseline won its score margin by 100; the half-size candidate lost by 259, a 359-point reversal.

Both programs issued the same commands through turn 14. On turn 15, the baseline starter moved outward toward a natural tree. The smaller program moved back toward home. On turns 18 and 19 it picked up and planted an apple, activating the compact orchard, while the baseline continued working the natural tree. The candidate then spent time waiting at and harvesting the orchard as the opponent expanded. This is the first strategically meaningful divergence in the worst loss.

That trace does *not* justify deleting the orchard. The no-orchard control lost 38.72 points on average and tripled loss mass. The evidence instead supports a narrower next question: **can orchard activation be made fail-closed and closer to the exact parent decision?** In plain language, the smaller bot should start an orchard only when it can prove that doing so will not abandon a more valuable natural-tree route or delay the economy at the wrong moment.

6. What is achieved, and what is not

Requirement	Status	Evidence
Source no larger than 31,410 bytes	✓ achieved	Several readable candidates are below the ceiling; newest is 31,407 bytes.
No renaming, compression or minification trick	✓ achieved	Candidates are built by named logical deletion and specialization; manifests explicitly record no identifier renaming or minification.
Compiles and emits valid commands	✓ achieved for newest	Optimized build, empty input and ten semantic fixtures pass.
Avoids known two-cell oscillations	✓ achieved for newest	Zero episodes of six or more alternating moves on development, fresh and exact-replay checks.
Preserves robust game value on unseen maps	✗ not yet	Every exact half-size hash has failed at least one frozen fresh-map gate or replay-safety check.
Mature Arena rank 15 or better	✗ not tested for a half-size bot	No candidate was safe enough to submit. The unchanged full-size live bot remains rank 11 in the latest recorded checkpoint.

The joint goal is therefore **not complete**. It would be misleading to call the 31,407-byte source “the new best bot”: it is the newest technically valid half-size research candidate, but it is transfer-rejected. The best current Arena bot remains the original full-size E7a.

7. Recommended next work

1. **Preserve the orchard but narrow its activation.** Compare the exact parent’s reason for continuing toward a natural tree with the compact policy’s reason for returning home and planting. Add the smallest readable condition that protects that choice.
2. **Keep the tree-aware liveness rule.** It removes long A-B oscillations and is better than the global no-backtrack rule. Do not spend bytes restoring a rule that the diagnostics have already shown to be weaker.
3. **Re-test on development and known failure traces first.** The candidate must keep second-worker training, avoid tent blockage, and not recreate the orchard collapse.
4. **Use a new untouched range only after the source is frozen.** The last fresh range is consumed. Reusing it can diagnose mechanisms but cannot provide honest transfer evidence.
5. **Submit only after every local gate passes.** Then recover the exact Arena source by hash and wait for enough live games to establish rank 15 or better.

8. Evidence and reproducibility appendix

The report is a plain-language synthesis of immutable task records and machine-readable results. The most important evidence locations are listed below. A SHA-256 value is a cryptographic fingerprint: if one byte changes, the fingerprint changes, so it identifies the exact source or result used.

Evidence		Repository-relative path or identifier
Chronological record	task	coordination/tasks/ 20260802-e7a-half-size-logical-simplification.md
Frozen test rules		docs/ e7a-half-size-logical-simplification-protocol-2026-08-02.md
Exact live baseline		cgauto/submissions/ candidate-agent6553250-preseed-e7a-lemon-near-tie.min.rs
Baseline fingerprint		97bfe71e3f2f05e1b8fa3c697c5e5db3 624ac9739e90954e9fa9be79a8e48595
Newest half-size source		local_codex_1/e7a-half-size-logical-simplification/ focused-yamo-bank-convoy-tree-edge-reversal.rs
Newest fingerprint		acbada47b9a3cf279ff5356a32e2965e b44cbb5ccc8d7b7e8c6f5dda3f92e847
Newest fresh result		data/analysis/live-agent-6553250/ e7a-half-size-tree-edge-reversal-fresh-result-2026-08-03.md
Worst-game comparison traces	compari-	local_codex_1/e7a-half-size-logical-simplification/ baseline-tree-edge-map9866014-seat0-gold-comparison.trace.tsv
No-orchard diagnostic		local_codex_1/e7a-half-size-logical-simplification/ focused-yamo-bank-convoy-door-harvest-no-orchard-consumed-9866-diagnostic.json

The protected resident source remained byte-exact throughout at SHA-256:

```
fff6669b0bc0b15b0992637f70c07197
e1838f403cb7fd038bc1fae73d52b13f.
```

No raw game cache was modified, no sealed map range was opened, and no Arena mutation occurred.

Prepared by protocol identity local_codex_1. Metrics are paired candidate-minus-baseline results. Diagnostic replays of already-open maps are explicitly not promotion evidence.